

Declaration

I, [Student Name], declare that this assignment, titled [Assignment Title], is my own original work and has not been copied from any other source except where explicitly acknowledged. I have not engaged in plagiarism, collusion, or any other form of academic misconduct in the preparation and submission of this assignment. All sources of information and data used in this assignment have been properly cited and referenced in accordance with the prescribed guidelines. I have not used unauthorized assistance in the preparation of this assignment and have not allowed any other student to copy my work. I am aware that any breach of academic integrity may result in disciplinary action as per the [policies of Monash University](#), which may include failing this assignment or the course, and further academic penalties.

Signature: Yang Tang

Date: 2/10/2026

Self-Evaluation

Rate your performance for each criteria. Put a ☒ (tick) in the box where you think your work belongs.

Criteria	Exceeds Expectations	Meets Expectations	Needs Improvement	Fail to meet expectations
Introduction	☒			
Main Body	☒			
Reflection		☒		
AI Acknowledgement		☒		

Introduction

Generative and agentic AI can help developers investigate performance problems, implement alternatives, and interpret results. These capabilities matter as JavaScript applications accumulate assets, dependencies, and rendering work. My WCIP practice involved a Vue gardening application with twelve plant records and fourteen photographs [1]. This report evaluates five strategies using that experience, while distinguishing implemented changes from unverified performance gains. AI supports investigation; developers and users must determine acceptable outcomes.

Main Body

Discovering Optimization Opportunities through Prototypes

AI can introduce developers and nondevelopers to unfamiliar optimization opportunities and translate suggestions into prototypes. In WCIP, Codex identified oversized images, theme CSS, external fonts, repeated data loading, and search recalculation [1]. It treated searching twelve records as a future concern, demonstrating why recommendations should reflect application scale.

The subsequent image prototype generated 39 WebP variants and introduced a reusable component. This provided a concrete implementation whose widths and compression settings could be refined. However, prototype availability does not establish either usability or performance. Additional generation steps create maintenance obligations, while convincing code can encourage inexperienced users to accept assumptions they cannot evaluate. AI therefore lowers the barrier to experimentation, but representative measurements and developer review remain necessary before adopting its proposals.

Implementing Hypotheses through Controlled Experiments

AI can rapidly implement a developer's optimization hypothesis and help design its evaluation. I requested shared plant data across views. Codex introduced `getPlants()`, retaining a shared promise, reusing parsed results, and allowing retries after failures [1]. This translated a specific idea into inspectable code rather than a general recommendation.

However, successful cache checks establish behaviour, not a perceptible speed improvement. HTTP caching might already reduce transfers, and twelve records provide little computational pressure. The implementation also retains data until reload, creating potential staleness if the catalogue changes. A controlled comparison should isolate this change, repeat identical navigation sequences, and distinguish cold from warm caches. AI can prepare such experiments, but developers must verify measurements instead of treating fewer function calls as proof of improved experience.

Investigating Existing Techniques and Their Applicability

AI can compare proposed approaches with documentation, existing implementations, and experimental research. My mipmap analogy prompted Codex to compare browser resizing, dynamic image services, and predefined responsive variants. Native `srcset` and `sizes` let browsers select suitable image resources [2]. For fourteen static photographs, generating variants during builds avoided the infrastructure of a dynamic service.

Nevertheless, this implements resolution selection rather than GPU mipmap filtering. Browser resizing would still require downloading the original. Likewise, findings from research papers require scrutiny of workloads and deployment assumptions before transfer to JavaScript applications. WCIP also illustrates hidden costs: its component and manifest add runtime content. Although code splitting can reduce initial JavaScript payloads [3], it already existed here; the image prototype demonstrated no JavaScript bundle reduction.

Diagnosing Bottlenecks from Structured Performance Evidence

AI can screen lengthy logs and summarize performance evidence, helping developers locate patterns and investigate anomalies. In WCIP, comparing the three homepage originals with their 320-pixel variants showed a reduction from 847,570 to 47,828 bytes, approximately 94.4% [1]. Such summaries make optimization candidates easier to assess.

However, this measures asset sizes, not page loading speed. Larger displays may select larger variants, and resizing and format conversion were not isolated. Codex reported successful build and asset checks, but browser automation was unavailable. Consequently, actual image selection and rendering improvements remained unverified. Future analysis should retain raw traces, identify long tasks, and compare Largest Contentful Paint and Interaction to Next Paint across repeated runs. Laboratory results can differ from real user experiences [\[4\]](#), so AI summaries require contextual interpretation.

Maintaining Improvements through Regression Monitoring

AI can help preserve improvements by drafting monitoring configurations, explaining regressions, and proposing targeted fixes. This is a proposed extension for WCIP, not completed work. Lighthouse CI can compare performance across changes and enforce predefined budgets [\[5\]](#). AI could help investigate whether a new dependency increases initial JavaScript or whether replacement photographs increase transfer costs.

However, conventional tools produce the measurements; AI interprets them. Automatically accepting AI fixes could introduce behavioural regressions, while noisy scores may encourage unnecessary changes. Developers should review repeated measurements, test cache recovery and image selection, and maintain explicit acceptance criteria. Users should inform acceptable compromises in clarity and responsiveness. Monitoring becomes useful when it detects meaningful deterioration, rather than rewarding higher scores regardless of product quality.

Reflection

WCIP suggests that AI's contribution lies in accelerating exploration and implementation while making alternatives easier to discuss. My prompts shaped the work, particularly the cache request and mipmap analogy. Yet choosing an implementation is different from establishing its value. No documented user study confirmed that these changes improved perceived responsiveness or preserved satisfactory image quality. The small catalogue also limits conclusions about larger applications.

Developers and users should jointly define success, balancing speed, accessibility, correctness, and maintenance. Performance logs should also be checked for sensitive information before sharing them with AI services. Future agentic workflows may automate more experiments, but their decisions should remain traceable to reproducible evidence. Human judgment is essential when metrics conflict with user needs.

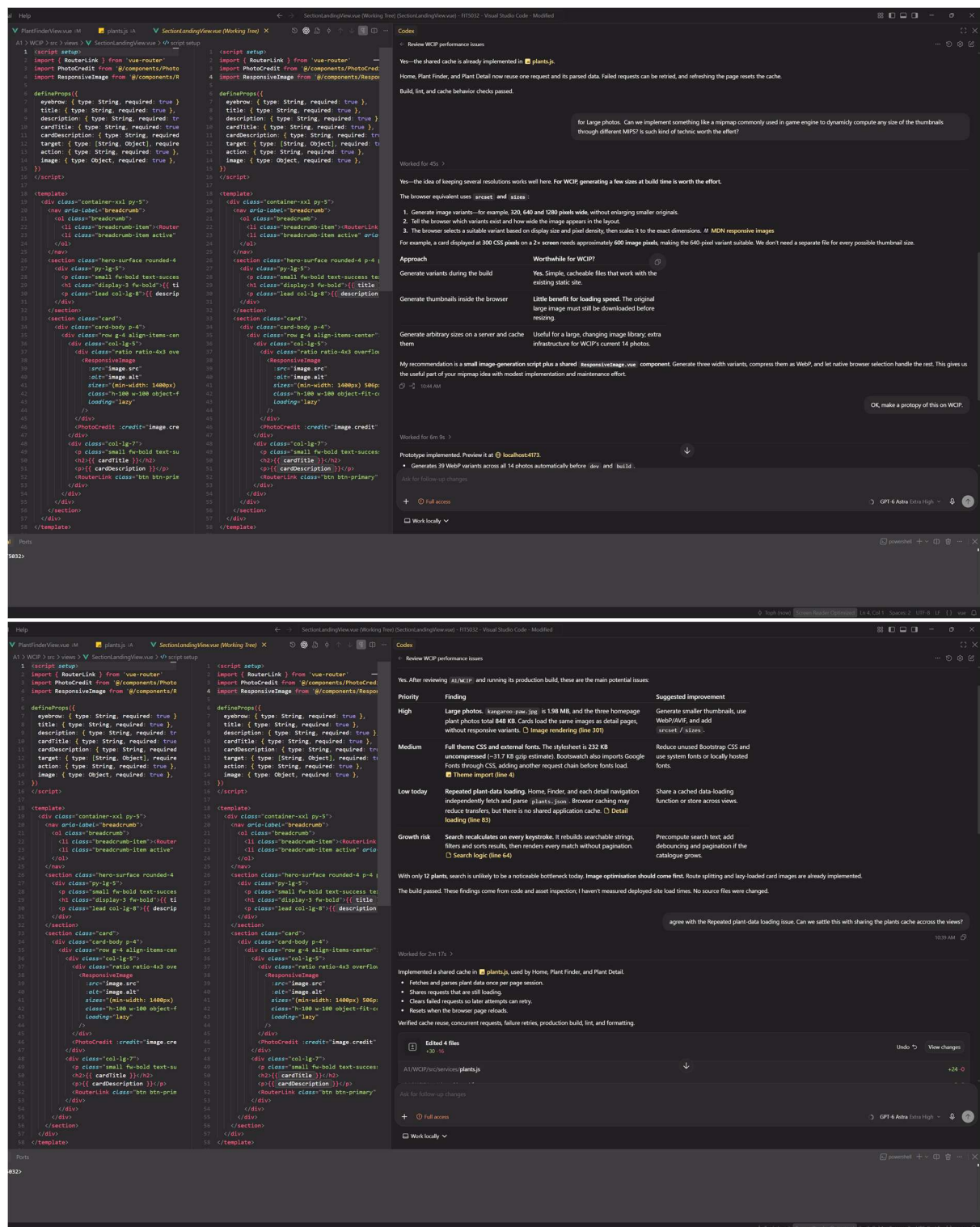
Conclusion

AI can support opportunity discovery, hypothesis testing, feasibility research, evidence analysis, and regression monitoring. WCIP demonstrates practical caching and image changes, while exposing gaps between implementation checks and validated performance. Further work should measure bundle effects, rendering behaviour, and user satisfaction across devices. AI should assist this process, with final standards and tradeoffs determined by developers and users.

Acknowledgement of AI use

I used Codex to review WCIP, implement shared caching, and prototype responsive images. I also used AI to examine the supplied evidence and help draft this report.

Practical Usage on site WCIP:



References

- [1] "WCIP source code and 'Review WCIP performance issues' interaction record," unpublished project materials supplied by the author, 2026.

- [2] MDN contributors, “Using responsive images in HTML,” *MDN Web Docs*. Accessed: Oct. 2, 2026. [Online]. Available: [MDN responsive images documentation](#).
- [3] H. Djirdeh and J. Wagner, “Reduce JavaScript payloads with code splitting,” *web.dev*. Accessed: Oct. 2, 2026. [Online]. Available: [Reduce JavaScript payloads with code splitting](#).
- [4] P. Walton, “Why lab and field data can be different (and what to do about it),” *web.dev*. Accessed: Oct. 2, 2026. [Online]. Available: [Why lab and field data can be different](#).
- [5] K. Hempenius, “Performance monitoring with Lighthouse CI,” *web.dev*. Accessed: Oct. 2, 2026. [Online]. Available: [Performance monitoring with Lighthouse CI](#).